

A2: Gaussian Splatting

Zeke Barnett
15-474

October 2026

AI Disclosure: **TODO:** write your own disclosure of how AI was used on this assignment.

Code: **TODO:** <https://github.com/<you>/15-474-A2>. The repository README gives the commands that regenerate every number and figure below.

Setup. PSNR uses $\text{MAX} = 1$ on float images. The 2D targets are the provided 256×256 images resized to 128×128 (Lanczos); every 2D fit uses the P3 initialization with a fixed seed, Adam at lr 10^{-2} , and 2000 steps of full-image MSE. The 3D fits use the provided 160×160 views, Adam at lr 10^{-2} , and 1500 steps with one random training camera per step. All runs are on an M-series Mac (mps).

Renderer. Compositing follows the handout: Gaussians sorted front to back, $C = \sum_i T_i \alpha_i c_i$ with $T_i = \prod_{j < i} (1 - \alpha_j)$, computed for all Gaussians at once with an exclusive `cumprod` instead of a Python loop. The first, fully dense version (every pixel against every Gaussian) needed about 1.25s per step at $N = 4096$, so a single P5 run took 40 minutes. Following the handout’s suggestion, the final renderer cuts the image into 16×16 tiles and composites each tile only against the Gaussians whose 3.5σ bounding box overlaps it. This is about $20\times$ faster (0.06s per step at $N = 4096$, two minutes per run). Contributions beyond 3.5σ (weight < 0.0022) are dropped; a unit test checks the tiled renderer against a dense reference with the same cutoff, and the cutoff moved coffee at $N = 256$ from 26.16 to 26.15 dB.

P4. Densification

Rule. Every 200 steps, until 75% of training (step 1500), I run a pass. The score g_i is the mean norm of $\partial L / \partial \mu_i$ over the steps since the last pass. I measure it in normalized device coordinates (the pixel gradient times $W/2, H/2$), which are the 3DGS paper’s units, so the threshold does not depend on image resolution; with raw pixel gradients the paper’s 2×10^{-4} selected nothing (the 99th percentile was about 6×10^{-5}). Gaussians with $g_i > 10^{-4}$ are densified. I lowered the threshold from the paper’s 2×10^{-4} because g_i shrinks as the fit improves, and at 2×10^{-4} growth stopped at 788 of a 1024 budget on astronaut.

- *Clone* if the Gaussian’s largest scale is at most 2% of the image width (2.6px): an exact copy is appended. The copy sits at a different place in the compositing order, so the two receive different gradients and separate.
- *Split* otherwise: the parent is replaced by two children whose centers are sampled from the parent’s own distribution, $\mu + RSz$ with $z \sim \mathcal{N}(0, I)$, with scale divided by 1.6 and the parent’s rotation, color, and opacity.

- *Prune* Gaussians with opacity below 0.005.
- *Budget*: each clone or split adds exactly one Gaussian, so after pruning, at most (budget – count) candidates are densified, highest g_i first.

New tensors are swapped into Adam after every pass: survivors keep their moment estimates and new Gaussians start from zero. A densified run starts from $N/4$ Gaussians. In 2D, pruning never fired: from the single viewpoint every Gaussian is visible, so the optimizer has no reason to make one transparent.



Figure 1: Astronaut at 128×128 . Left to right: target, fixed-count fit with 1024 Gaussians (27.47 dB), densified fit grown from 256 to 1024 Gaussians (**29.88 dB**).

Densified vs. fixed. On astronaut, with both runs ending at exactly 1024 Gaussians, densification reaches 29.88 dB against 27.47 dB for the fixed-count fit, a 2.4 dB gain (Figure 1): the flag stripes, mission patches, and the helmet and shuttle edges are visibly sharper. With a fixed count, the Gaussians mostly stay near their uniformly random starting positions (at $\text{lr } 10^{-2}$ a center moves at most about 0.01 px per step), so smooth regions get as many Gaussians as edges. Densification puts new Gaussians where the gradient says the fit is failing, which is along the edges. Densified runs are also faster, since they spend most of training below the budget. Densified runs at other budgets are on the dashed curves of Figure 2; most of them stopped short of their budget, which I discuss there.

P5. Quality vs. number of Gaussians

target	$N = 256$	$N = 1024$	$N = 4096$
coffee	26.15	31.88	38.21
astronaut	21.65	27.47	36.93
cat	27.38	31.56	40.72
values stored ($9N$) / raw ($128^2 \cdot 3$)	0.047	0.19	0.75

Table 1: Final PSNR (dB) of fixed-count fits. Same initialization and seed for every run; only N changes.

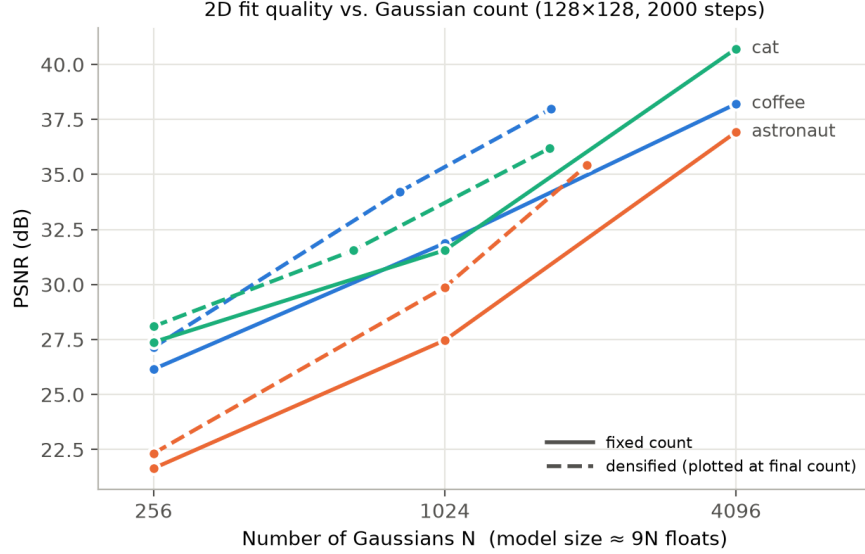


Figure 2: PSNR against N (log scale). Solid: fixed count (Table 1). Dashed: densified runs, plotted at the count they actually ended with.

Astronaut needs the most Gaussians, and is the lowest curve at every N , 4–6 dB behind the others at 256 and 1024. Its difficulty is structure rather than texture: flag stripes, the white suit against a black background, and lettering are long, sharp, high-contrast edges. A Gaussian has a soft profile, so representing an edge takes a row of small Gaussians along it, and every pixel the edge is wrong on costs a large squared error.

Coffee is the smooth case. Large flat regions (saucer, table, coffee surface) are covered by a few large Gaussians, so it does well at low N . Its curve flattens toward 4096 because the remaining error is concentrated in a few sharp edges, the cup rim and the spoon.

Cat is easier than its frequency content suggests. I expected the fur to be the hardest target, but at 128×128 it is the best at 4096 and close to coffee below that. Two reasons. Resizing from 256 to 128 removes the finest fur strands before fitting starts. And what remains is mostly low-contrast texture: the error is spread over many pixels but each error is small, which PSNR penalizes gently, while the high-contrast features (eyes, nose) are compact. I expect cat would fall behind at its native resolution. The steep jump from 1024 to 4096 also reflects that $N = 4096$ is barely compression: $9N = 36,864$ stored values against 49,152 raw pixel values, about one Gaussian per four pixels, at which point the frequency content of the image matters much less.

Densified curves. Densification is ahead at every matched count (all three images at 256, astronaut at 1024). With a fixed threshold, though, growth stalls once the fit is good: coffee and cat stopped at 827 and 663 of 1024, and all three 4096 runs stopped between 1688 and 2014. Even so, densified coffee reaches 37.99 dB with 1699 Gaussians against 38.21 dB for 4096 fixed ones, nearly the same quality with 41% of the Gaussians. A top- k fallback would force every run to its budget.

P7. 3D reconstruction

Setup. 4000 Gaussians, initialized exactly as in the handout: centers uniform in $[-1.5, 1.5]^3$, isotropic scale 0.08, identity quaternion, gray color, opacity ≈ 0.12 . Each step picks a random training view, projects every Gaussian (P6: mean through the pinhole, covariance through $JR_{wc}\Sigma R_{wc}^\top J^\top$), skips Gaussians within 0.2 of the camera plane, sorts the rest by camera depth z_c nearest first, and composites with the tiled renderer. Following 3DGS, 0.3 px^2 is added to each projected covariance so no splat is thinner than about a pixel. Training took 137 s.

Result. Mean PSNR over the 44 training views is **23.67** dB (mean of per-view PSNR). The spheres are in the right places with the right colors, but soft, and the background is a noisy haze crossed by dark streaks (Figure 3). Most Gaussians end near their initial opacity (median 0.11); at $\text{lr } 10^{-2}$ an opacity logit moves slowly, so the spheres are built from many faint blobs, and the bulk of the Gaussians drift outward (median $\|\mu\| = 1.75$) to paint the background.

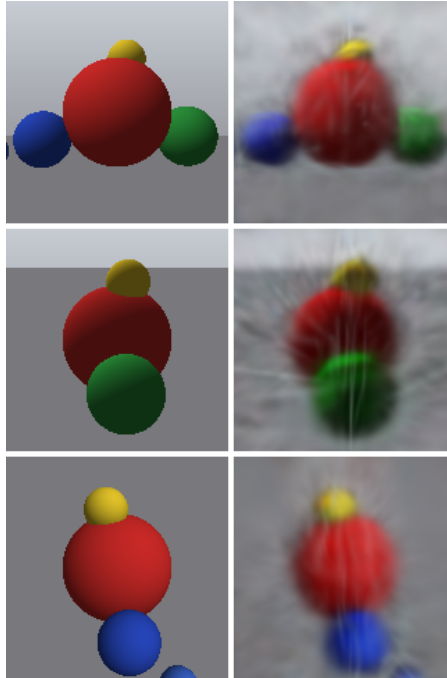


Figure 3: Training views 000, 015, 033: ground truth (left) and the plain 4000-Gaussian fit (right).

P8. Densification in 3D

What changed and what stayed the same. The clone/split/prune logic and the Adam bookkeeping are the same code as P4; I generalized **densify** so the caller passes a size limit in its own units and a function returning each Gaussian’s rotation matrix. What changed: means and scales are 3D and split offsets are sampled in 3D, $\mu + RSz$ with R from the quaternion (clones and children copy the quaternion); the size threshold is 0.1 world units instead of a fraction of the image; and the score is the world-space position gradient, averaged over the random cameras since the last pass. Passes run every 100 steps until step 1200, starting from 1000 Gaussians with a 4000 budget and threshold 10^{-4} (more than half of the Gaussians exceed it early in training, fewer later).

Pruning needed an opacity reset. The handout identifies pruning as what removes floaters, but the floaters in the P7 fit sit around opacity 0.1, twenty times the 0.005 threshold, and a first densified run pruned nothing. So I added the 3DGS paper’s opacity reset: every 400 steps (at 400 and 800) every opacity is capped at 0.01 and Adam’s opacity moments are cleared. Gaussians that the training views need grow back; haze that only explains individual views stays faint and can be pruned.

run	N	train PSNR	held-out PSNR
plain (P7)	4000	23.67	16.43
densified	4000	23.82	18.76
<i>ablations</i>			
densified, no opacity reset	4000	24.87	15.91
densified, prune threshold 0.01	3969	23.84	18.58

Table 2: 3D fits, 1500 steps each. Two identical runs differ by about 0.2dB held-out (MPS is not deterministic).

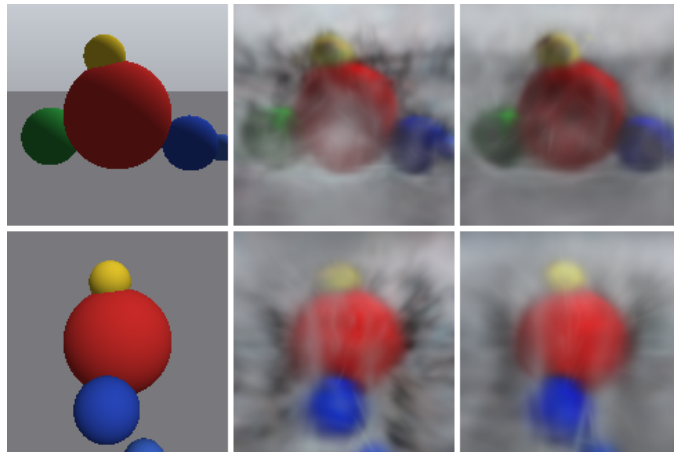


Figure 4: Held-out views 002 and 007. Left to right: ground truth, plain fit, densified fit. The dark streaks (floaters) of the plain fit are mostly gone.

Result. At the same count, the densified fit gains 2.3 dB on held-out views while training PSNR is essentially unchanged (Table 2), and most of the floaters are visibly gone (Figure 4). The ablations show where the gain comes from. Densifying *without* the reset overfits more than the plain fit: higher training PSNR, lower held-out PSNR, because the new Gaussians are spent explaining individual views. The reset is what removes the floaters, by making every Gaussian re-earn its opacity against all views. Pruning itself removed nothing at the 0.005 threshold, and raising the threshold to 0.01 removed only 14 Gaussians and changed nothing beyond run-to-run noise. In a 1500-step schedule, then, the reset does the work by turning floaters nearly transparent, not by deleting them; a longer schedule would give the faint ones time to fall below the threshold.

P9. Novel-view evaluation

held-out elevation	5°	25°	45°
plain	14.74	17.54	17.02
densified	17.65	17.94	20.71

Table 3: Mean held-out PSNR (dB) over the four views at each elevation. Training orbits are at -5° , 15° , 35° , 55° .

Held-out vs. training. Held-out PSNR is 16.43 dB for the plain fit against 23.67 dB on training views, a 7.2 dB gap; for the densified fit it is 18.76 against 23.82 dB, a 5.1 dB gap. The gap is the part of the training fit that is memorization rather than reconstruction.

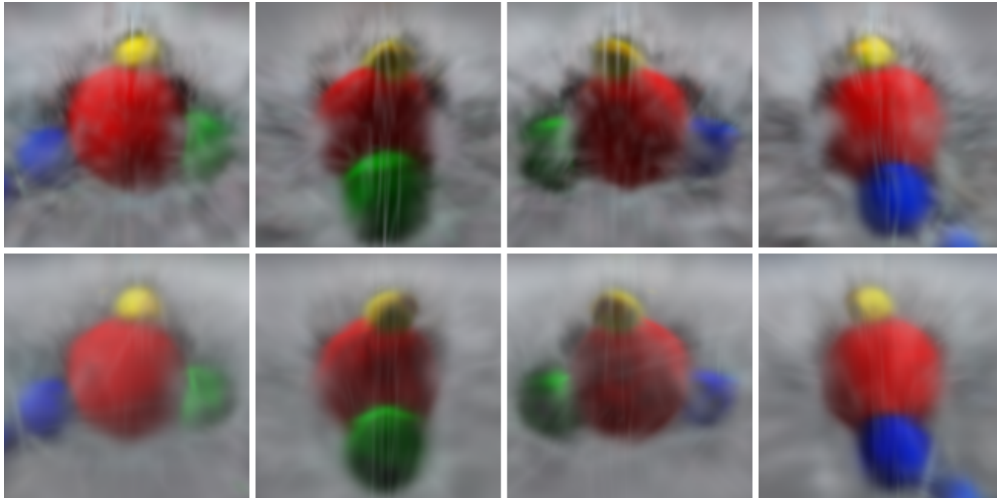


Figure 5: Orbit at 25° elevation, between the training orbits, azimuth 0° , 90° , 180° , 270° . Top: plain fit; bottom: densified fit. The full 36-frame orbits are `results/3d/plain/orbit.gif` and `results/3d/densify/orbit.gif` in the repository.

Where it breaks down.

- **The background** is the largest error. The ground truth has a sharp horizon between a light sky and a dark floor that extends to infinity. Gaussians near the origin cannot represent anything at infinity consistently, so each training view gets its own painted haze, which is wrong from any other angle. This is why the 5° views, where the horizon crosses the frame, are worst (Table 3), and the 45° views, which see mostly uniform floor, are best for the densified fit.
- **Floater**s: the dark streaks of the plain fit are elongated, faint Gaussians at wrong depths that help some training views and cut across novel ones. The opacity reset removes most of them (Figure 4).
- **Thin vertical needles** remain in the orbit (Figure 5). The training cameras lie on only four elevation rings, so a Gaussian stretched vertically is seen nearly end-on or blurred into the spheres from most training views, and little in the loss penalizes the stretch. From the 25° orbit it shows up as a bright line.

- **Softness everywhere.** At $\text{lr } 10^{-2}$ opacities grow slowly, so solid spheres are approximated by many semi-transparent blobs, and their edges blur. The per-parameter learning rates used by 3DGS (much higher for opacity, much lower for position) would likely sharpen this; I kept the handout's single learning rate so the P7 result matches the specification.